

HONEYWELL EMBEDDED ENGINEERING TEAMS — PA · IA · BA

AI Champions Programme

Module 04

Spec-Driven Development for Embedded Software

From requirement to traceable implementation — spec → plan → tasks → implementation, with GitHub Spec Kit / OpenSpec turning context-engineering discipline into a repeatable workflow.



DURATION

2 Hours · Module 4 of 12



FORMAT

Guided Build + Hands-On Spec Lab



DELIVERED FOR

Honeywell Embedded Engineering

How We'll Spend These Two Hours

Six connected moves — from SDD fundamentals to a fully traceable, validated implementation.



01

SDD Fundamentals & The Spec Flow

15 min

Spec → plan → tasks → implementation — and why this beats prompt-only development.



02

What Goes Into a Specification

20 min

Requirements, constraints, interfaces/contracts & acceptance criteria — the anatomy of a good spec.



03

Write a Specification, Live

25 min

Author a spec for a real embedded feature using GitHub Spec Kit / OpenSpec patterns.



04

Generate Plan & Tasks

20 min

Turn the specification into a traceable plan and implementation-ready tasks.



05

Implement Against the Spec

25 min

Implement with Copilot, constantly checked back against requirements and constraints.



06

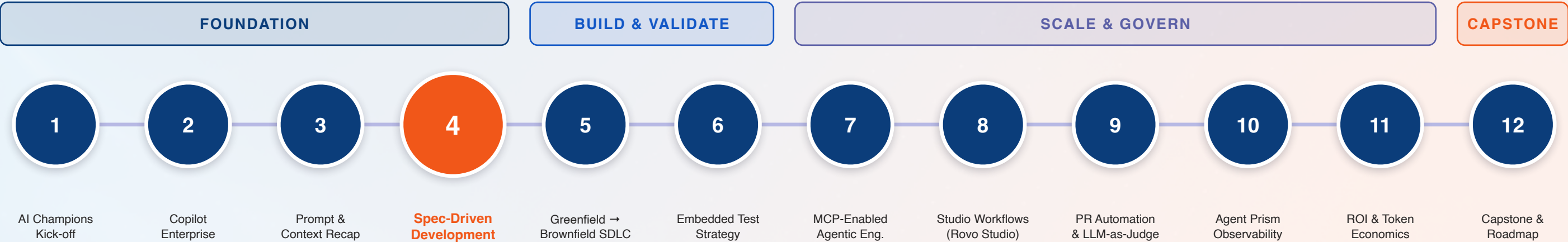
Validate Traceability

15 min

Prove every acceptance criterion maps to an implemented, tested behaviour.

Module 04 in the 12-Module Journey

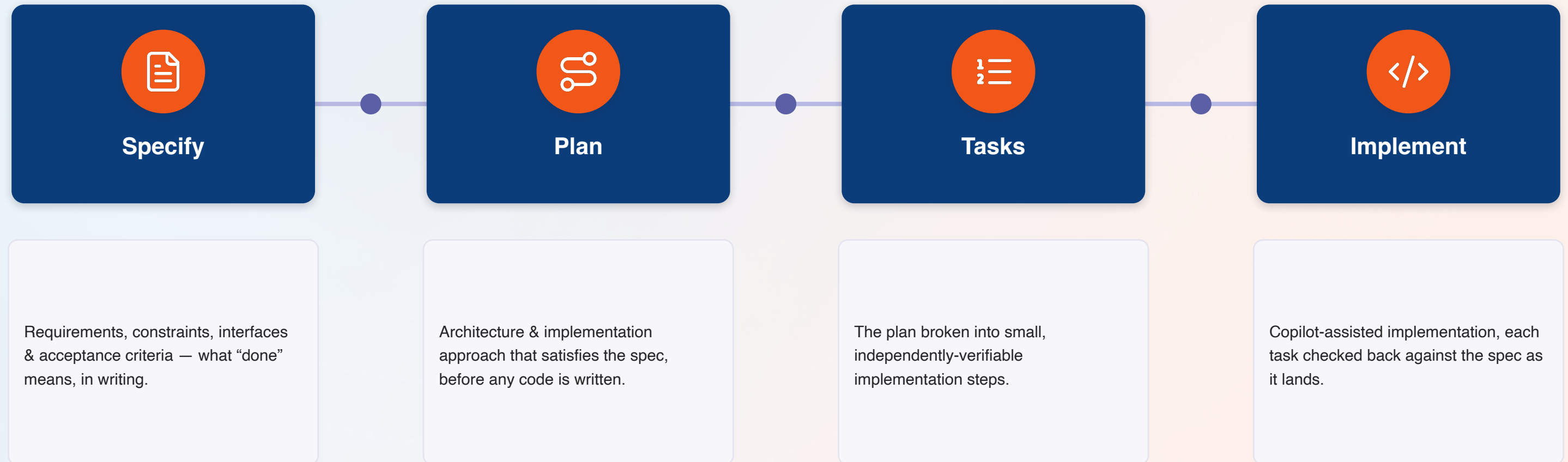
Context engineering applied to a full feature — Module 05 takes this spec workflow through a complete SDLC.



Why this matters: The greenfield and brownfield work in Module 05, the test strategy in Module 06, and the PR quality gates in Module 09 all validate against the specification this module teaches you to write — traceability starts here.

SDD Fundamentals: Spec → Plan → Tasks → Implement

Four stages, each one validating the next — the backbone of every module from here forward.



The rule that makes this work: Every stage traces back to the one before it. A task with no line back to a requirement is a task worth questioning.

What Goes Into a Specification

Four categories, one traceable document — the anatomy of a spec worth building from.



Requirements

Functional requirements

Non-functional requirements

Stakeholder intent



Constraints

Timing / resource constraints

Memory & performance limits

Platform & toolchain limits



Interfaces & Contracts

Function signatures & structs

API / driver boundaries

Existing behaviour to preserve



Acceptance Criteria

Observable pass/fail conditions

Edge & negative cases

Scoped, not exhaustive: A good spec covers exactly what this feature needs from each category — the same task-scoping discipline from Module 03, applied to a document instead of a prompt.

What the Calibration Survey Told Us About Spec Discipline Today

Real answers from 19 Honeywell engineers — most of this cohort is already closer to SDD than “prompt-only.”

How a Requirement Becomes Working Code

15

of 19

• Formal spec→plan→tasks (8)

• Spec & design→tests (7)

• Prompt / discussion→code (4)

15 of 19 — 79% — already work from some form of specification before implementation. This module gives that practice a name, a tool, and a traceable structure.

Only 3 of 19 explicitly asked to “learn specification-driven development” as a training goal — it isn't unfamiliar, it's under-tooled. GitHub Spec Kit changes that.

How Does Your Team Specify New Features?

User stories or tickets

8

A mix of the above

5

Requirements documents

4

Technical specifications

2

The gap this module closes: User stories and tickets rarely carry acceptance criteria or traceability — today's spec adds exactly that layer, without slowing teams down.

Prompt-Only vs. Spec-Driven Development

The same distinction from Module 03, now applied to a whole feature instead of a single task.

PROMPT-ONLY DEVELOPMENT

- “Add a feature that does X” — acceptance criteria exist only in the requester's head.
- No plan step: Copilot jumps straight from request to code, right or wrong.
- Interfaces & constraints get discovered by trial and error, mid-implementation.
- No traceability — impossible to prove later that the requirement was actually met.
- Review has to re-derive intent from a diff, since there's no spec to check against.



SPEC-DRIVEN DEVELOPMENT

- Acceptance criteria are written down before implementation starts — observable, testable.
- A plan step validates the approach against constraints before any code is generated.
- Interfaces & constraints are stated up front, scoped exactly like Module 03's context layers.
- Every task traces back to a requirement — traceability is a byproduct, not extra work.
- Review checks the diff against the spec directly — faster, more objective sign-off.

Specification Quality: What “Good” Looks Like

Five checks that separate a traceable spec from a wish list.



Testable

Every requirement resolves to a pass/fail condition — no room for “works correctly” as an acceptance criterion.



Bounded

Constraints are explicit: timing, memory, resource limits — stated as numbers, not adjectives.



Interface-Accurate

Function signatures and contracts match what the codebase actually exposes, not an idealized version.



Traceable

Each requirement carries an identifier a task and a test can reference back to.



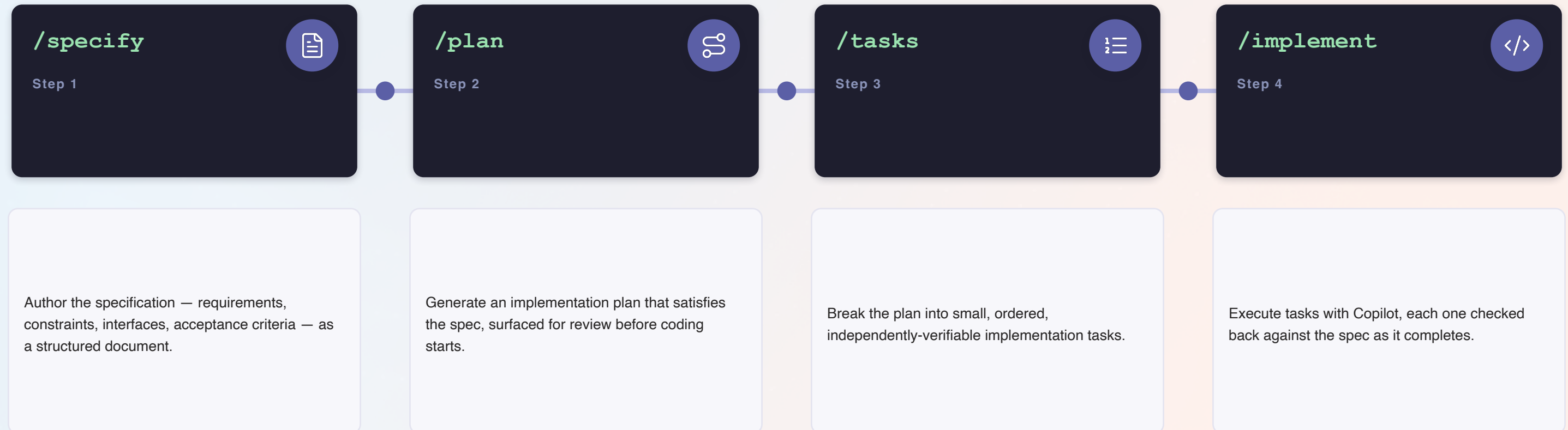
Reviewed

A second engineer has checked the spec itself — before implementation, not just the code after.

Quick test: If you can't write the acceptance-criteria checklist from the spec alone, the spec isn't done yet.

GitHub Spec Kit / OpenSpec: The Workflow in Practice

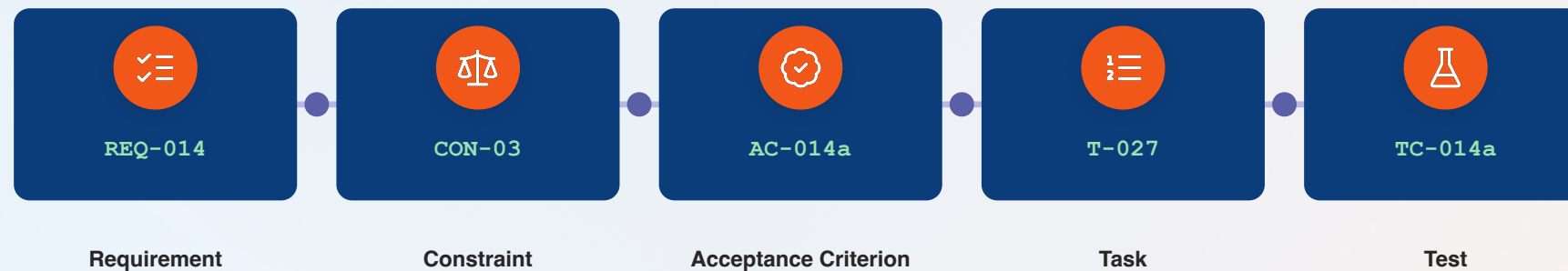
Four commands, one traceable artifact chain — the tooling behind today's spec flow.



Tool-agnostic principle: Whether it's GitHub Spec Kit or an OpenSpec-equivalent pattern, the four-stage discipline — not the specific CLI — is what this module teaches.

Traceability & Change Control

Every implementation decision should answer one question: which requirement made me necessary?



When the Spec Changes Mid-Flight

Update the requirement first, then let the change ripple forward through constraint → acceptance criterion → task → test. Never patch a task without touching the spec it traces to.

Why this matters for Module 09: LLM-as-Judge PR review checks a change against exactly this chain — specification compliance is only checkable if the chain exists to check against.

Example: One Requirement, Traced End to End

REQ-014 “Diagnostic messages must include a monotonic timestamp” → CON-03 “formatter must not allocate on the heap” → AC-014a “timestamp field present and increasing across 100 consecutive messages” → T-027 “implement timestamp field in diag_format()” → TC-014a “unit test asserting monotonic timestamps under load.”

Acceptance Criteria in Practice: Weak vs. Strong

The same feature request, specified two ways — only one of them is implementable without guesswork.

WEAK ACCEPTANCE CRITERIA

"The sensor-polling loop should handle errors gracefully and not impact performance."

- ✗ "Gracefully" has no test
- ✗ "Not impact performance" has no threshold
- ✗ No stated polling interval or timeout behaviour
- ✗ Nothing to check the implementation against

STRONG ACCEPTANCE CRITERIA

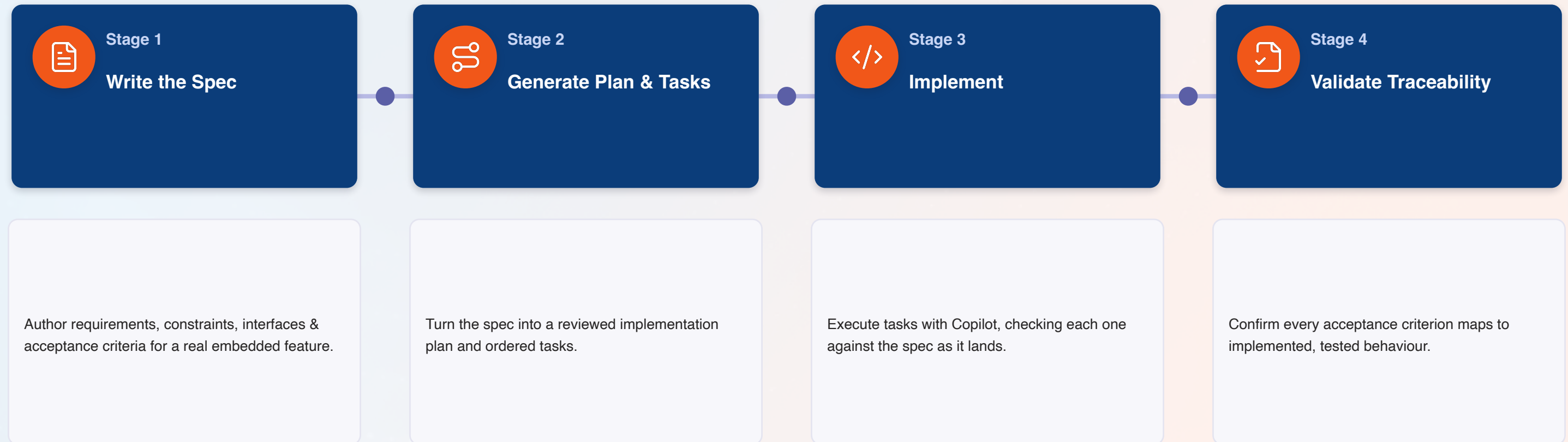
"AC-1: On sensor read timeout (>50ms), retry once then log SENSOR_TIMEOUT and return last-known value. AC-2: Polling loop completes within 2ms at 100Hz, measured over 1,000 cycles."

- ✓ Explicit threshold: 50ms timeout, 2ms budget
- ✓ Named, checkable behaviour on failure
- ✓ Directly convertible into a unit test

Notice: The strong version reads like a test case because it's meant to become one — that's the acceptance-criteria quality bar from Slide 8, applied.

Hands-On Lab: Spec to Validated Implementation

Today's arc, in four stages — one embedded feature, fully traced from requirement to test.



Bring forward: The context layers from Module 03 and the specification you write today both carry directly into Module 05's greenfield build.

Anti-Patterns: How Spec-Driven Development Goes Wrong

Five habits that quietly turn a spec into paperwork nobody actually uses.

DO THIS

- ✓ Write acceptance criteria as testable, observable conditions
- ✓ Update the spec first when scope changes mid-implementation
- ✓ Keep the plan step even for small features
- ✓ Review the spec itself before implementation starts
- ✓ Let acceptance criteria double as the test list

NOT THAT

- ✗ Write acceptance criteria as vague adjectives (“fast,” “robust”)
- ✗ Patch the code and leave the spec stale and misleading
- ✗ Skip straight from spec to code “to save time”
- ✗ Only review the diff, after the fact
- ✗ Write acceptance criteria and tests as unrelated afterthoughts

Module 04 Outcomes & What's Next

Everything below should be true before we start the full greenfield SDLC in Module 05.



Can explain the spec → plan → tasks → implementation flow and why each stage validates the next



Wrote a real specification using GitHub Spec Kit / OpenSpec patterns



Generated a plan and task breakdown directly from a specification



Understand the change-control discipline: update the spec first, let it ripple forward



Know what belongs in a specification: requirements, constraints, interfaces, acceptance criteria



Can tell weak acceptance criteria from strong, testable ones on sight



Implemented against the spec and checked traceability end to end



Know the five anti-patterns that turn SDD into paperwork nobody uses



NEXT — MODULE 05: EMBEDDED DEVELOPMENT — SDD TO COMPLETE SOFTWARE SDLC

3.5 hours · Build a greenfield feature from spec through implementation and testing, then apply the same discipline safely to a brownfield firmware repository.

Questions & Discussion

Module 04 — Spec-Driven Development for Embedded Software